

- Description

This module manages a smart reservoir flood control and water resource optimization system, ensuring that:

1. **Water levels are continuously monitored to prevent flooding and ensure water availability.**
2. **If the water level exceeds the flood risk threshold, controlled drainage is activated to release excess water.**
3. **If the water level drops below the minimum required level, water conservation measures are implemented.**
4. **A log of all water level changes, drainage actions, and conservation measures is maintained for resource planning.**

This system helps prevent floods, optimize water distribution, and ensure sustainable reservoir management.

- Specification

module SmartReservoirFloodControl

var currentWaterLevel, maxWaterLevel, minWaterLevel: real

var drainageActive: boolean

var log: sequence of string

process updateWaterLevel(changeAmount: real)

functional scenario form:

currentWaterLevel <= maxWaterLevel && log = log $\oplus$ ("Water Level Updated: " + currentWaterLevel + "m, Status Stable."), Water level updated successfully.

||

currentWaterLevel > maxWaterLevel && drainageActive = true, log = log $\oplus$ ("Flood Warning: " + currentWaterLevel + "m, Activating Controlled Drainage."), Flood warning triggered, drainage activated.

||

currentWaterLevel < minWaterLevel && drainageActive = false, log = log $\oplus$ ("Water Shortage Alert: " + currentWaterLevel + "m, Implementing Conservation Measures."), Water shortage alert triggered, conservation measures applied.

end process

end module

- Java Code

```
package code;
import java.util.*;
class SmartReservoir {
    private double maxWaterLevel;
    private double minWaterLevel;
    private double currentWaterLevel;
    private boolean drainageActive;
    private List<String> log;
```

```

    public SmartReservoir(double maxWaterLevel, double minWaterLevel, double initialWaterLevel) {
        this.maxWaterLevel = maxWaterLevel;
        this.minWaterLevel = minWaterLevel;
        this.currentWaterLevel = initialWaterLevel;
        this.drainageActive = false;
        this.log = new ArrayList<>();
    }
    public void updateWaterLevel(double changeAmount) {
        currentWaterLevel += changeAmount;
        manageFloodControl();
        String message = String.format("Current Water Level: %.2f meters", currentWaterLevel);
        if (currentWaterLevel > maxWaterLevel) {
            message += " | WARNING: Flood Risk! Activating Controlled Drainage.";
        } else if (currentWaterLevel < minWaterLevel) {
            message += " | ALERT: Water Shortage! Implementing Conservation Measures.";
        } else {
            message += " | Water Level is Stable.";
        }
        System.out.println(message);
        log.add(message);
    }
    private void manageFloodControl() {
        if (currentWaterLevel > maxWaterLevel) {
            drainageActive = true;
        } else {
            drainageActive = false;
        }
    }
    public void printLog() {
        System.out.println("\nReservoir Water Level Log:");
        for (String logEntry : log) {
            System.out.println(logEntry);
        }
    }
}
public class c55_SmartReservoirSystem {
    public static void main(String[] args) {
        SmartReservoir reservoir = new SmartReservoir(50.0, 10.0, 30.0);
        // testcase-VT:

```

```
//      reservoir.updateWaterLevel(15.0);
//      reservoir.updateWaterLevel(-20.0);
//      reservoir.updateWaterLevel(-18.0);
//      reservoir.updateWaterLevel(20.0);
//      reservoir.updateWaterLevel(30.0);

//      // testcase-FT:
//      reservoir.updateWaterLevel(0.0);
//      reservoir.updateWaterLevel(-1.7976931348623157E308);
//      reservoir.updateWaterLevel(1.6349922841177022E296);
//      reservoir.updateWaterLevel(1.0715086069397108E301);
//      reservoir.updateWaterLevel(6.362033346062261E293);

//      // testcase-Z3:
//      reservoir.updateWaterLevel(15);
//      reservoir.updateWaterLevel(-20);
//      reservoir.updateWaterLevel(-18);
//      reservoir.updateWaterLevel(20);
//      reservoir.updateWaterLevel(30);
//      reservoir.printLog();

//      // testcase-UVT:
//      reservoir.updateWaterLevel(15.0);
//      reservoir.updateWaterLevel(-20.0);
//      reservoir.updateWaterLevel(-18.0);
//      reservoir.updateWaterLevel(20.0);
//      reservoir.updateWaterLevel(30.0);
//      reservoir.printLog();
    }
}
```

- Coverage report

1. Test case number: 5
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.0 %	8	834	842
✓ src	1.0 %	8	834	842
✓ code	1.0 %	8	834	842
✓ c55_SmartReservoirSystem.java	100.0 %	8	0	8
> c55_SmartReservoirSystem		0	0	0
> SmartReservoir	100.0 %	8	0	8

3. Vibration testing instruction coverage: 97.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.6 %	129	20,128	20,257
✓ src	0.6 %	129	20,128	20,257
✓ code	0.6 %	129	20,128	20,257
✓ c55_SmartReservoirSystem.java	97.7 %	129	3	132
> c55_SmartReservoirSystem	89.3 %	25	3	28
> SmartReservoir	100.0 %	104	0	104

4. Fuzz testing branch coverage: 75%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.7 %	6	836	842
✓ src	0.7 %	6	836	842
✓ code	0.7 %	6	836	842
✓ c55_SmartReservoirSystem.java	75.0 %	6	2	8
> c55_SmartReservoirSystem		0	0	0
> SmartReservoir	75.0 %	6	2	8

5. Fuzz testing instruction coverage: 91.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.6 %	121	20,136	20,257
✓ src	0.6 %	121	20,136	20,257
✓ code	0.6 %	121	20,136	20,257
✓ c55_SmartReservoirSystem.java	91.7 %	121	11	132
> c55_SmartReservoirSystem	89.3 %	25	3	28
> SmartReservoir	92.3 %	96	8	104

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.0 %	8	834	842
✓ src	1.0 %	8	834	842
✓ code	1.0 %	8	834	842
✓ c55_SmartReservoirSystem.java	100.0 %	8	0	8
> c55_SmartReservoirSystem		0	0	0
> SmartReservoir	100.0 %	8	0	8

7. Z3 instruction coverage: 97.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.6 %	129	20,128	20,257
✓ src	0.6 %	129	20,128	20,257
✓ code	0.6 %	129	20,128	20,257
✓ c55_SmartReservoirSystem.java	97.7 %	129	3	132
> c55_SmartReservoirSystem	89.3 %	25	3	28
> SmartReservoir	100.0 %	104	0	104

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.0 %	8	834	842
✓ src	1.0 %	8	834	842
✓ code	1.0 %	8	834	842
✓ c55_SmartReservoirSystem.java	100.0 %	8	0	8
> c55_SmartReservoirSystem		0	0	0
> SmartReservoir	100.0 %	8	0	8

9. Unimproved V-method instruction coverage: 97.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.6 %	129	20,128	20,257
✓ src	0.6 %	129	20,128	20,257
✓ code	0.6 %	129	20,128	20,257
✓ c55_SmartReservoirSystem.java	97.7 %	129	3	132
> c55_SmartReservoirSystem	89.3 %	25	3	28
> SmartReservoir	100.0 %	104	0	104